EE660 Lecture Notes: Perceptron Algorithm and Test Error Bound

Stephen Tu

August 19, 2026

1 Supervised learning: classification

We consider a distribution $\mathcal{D}$ over an event space $\mathsf{Z} = \mathsf{X} \times \mathsf{Y}$. Here, the space X denotes the *covariates*, and we will typically assume for simplicity that X is a subset of Euclidean space. On the other hand, the space Y denotes the *labels*. We will let $p(x, y) = p(x)p(y \mid x)$ denote the joint density function over Z.[1]

We first study *binary classification*, where $\mathsf{Y} = \{-1, +1\}$. Let the *training data* $S_n := \{(x_1, y_1), \ldots, (x_n, y_n)\}$ be n iid draws from $\mathcal{D}$.

Goal: Binary Classification. Using the training data S_n, learn a **predictor**

$$\hat{f}_n : \mathsf{X} \to \{-1, +1\}$$

with small **test error**:

$$\mathbb{P}_{(x,y)\sim\mathcal{D}}\left(\hat{f}_n(x) \neq y\right).$$

1.1 What form should the predictor take?

In order to propose an algorithm to learn the predictor $\widehat{f}_n$, we need to specify two things:

(a) What form should the predictor function $\widehat{f}_n$ take?

(b) What algorithm should we use to learn its representation?

To shed some insight into question (a), let us set forth a framework to study what an *optimal* predictor looks like. Let $\ell : \mathsf{Y} \times \mathsf{Y} \mapsto \mathbb{R}$ be a loss function, where we think of $\ell(\widehat{y}, y)$ as the cost of predicting $\widehat{y}$ when the true label is y. For a concrete example, think of $\ell(\widehat{y}, y) = \mathbf{1}\{\widehat{y} \neq y\}$, which is the *zero-one loss* or the *mistake loss*.

With this setup, we can now pose an optimization problem to compute the best predictor:

$$\min_{f:\mathsf{X}\to\mathsf{Y}} \mathbb{E}_{(x,y)\sim\mathcal{D}}[\ell(f(x), y)]. \tag{1.1}$$

Above, the optimization is taken over all (measurable) functions mapping covariates X to labels Y. It may seem at first that this is an impossible problem to solve, but it actually has a very simple solution.

Proposition 1.1. *Suppose the loss ℓ is* proper*, i.e.,*

$$\ell(1, -1) > \ell(-1, -1), \quad \textit{and} \quad \ell(-1, 1) > \ell(1, 1).$$

A solution $\widehat{f} : \mathsf{X} \to \mathsf{Y}$ of Equation (1.1) takes on the form:[2]

$$\widehat{f}(x) = \operatorname{sgn}\left\{p(y = +1 \mid x) - \frac{\ell(1, -1) - \ell(-1, -1)}{\ell(-1, 1) - \ell(1, 1)} p(y = -1 \mid x)\right\}. \tag{1.2}$$

[1] We will not be overly concerned with measure-theoretic concerns in this course.

[2] The sgn function is $\operatorname{sgn}(a) := \mathbf{1}\{a \geqslant 0\} - \mathbf{1}\{a < 0\}$. Note the tie at $a = 0$, such that $\operatorname{sgn}(0) = 1$, is broken arbitrarily.

Proof. By iterated expectations,

$$\mathbb{E}[\ell(f(x), y)] = \mathbb{E}[\mathbb{E}[\ell(f(x), y) \mid x]].$$

Let us now study the inner quantity, conditioned on x. The key is that $f(x)$, once conditioned on x, is no longer random (the jargon I will often use is that $f(x)$ is "x-measurable"). Hence, the inner conditional expectation decomposes very cleanly:

$$\mathbb{E}[\ell(f(x), y) \mid x] = \ell(f(x), 1)p(y = +1 \mid x) + \ell(f(x), -1)p(y = -1 \mid x).$$

Hence, an optimal $f(x)$, *for this value of* x that we are conditioning on, should pick either $+1$ or -1 to minimize the RHS of the above equation. That is, abbreviating $p_1 = p(y = 1 \mid x)$ and $p_{-1} = p(y = -1 \mid x)$:

$$\begin{aligned} f(x) &= \arg\min_{y \in \{\pm 1\}} \{\ell(y, 1)p_1 + \ell(y, -1)p_{-1}\} \\ &= \begin{cases} +1 & \text{if } \ell(-1, 1)p_1 + \ell(-1, -1)p_{-1} \geqslant \ell(1, 1)p_1 + \ell(1, -1)p_{-1} \\ -1 & \text{otherwise.} \end{cases} \end{aligned}$$

The result now follows by re-arranging the expression above, relying on the properness of the loss ℓ to divide by $\ell(-1, 1) - \ell(1, 1)$. □

Let us now consider a special case of Equation (1.2) where $\ell(\widehat{y}, y) = \mathbf{1}\{\widehat{y} \neq y\}$ is the zero-one loss. In this case, the predictor $\widehat{f}$ reduces to the *maximum a-posterior* (MAP) predictor:

$$\widehat{f}(x) = \arg\max_{y \in \mathsf{Y}} p(y \mid x). \tag{1.3}$$

Note that by Bayes' rule, we have that:

$$p(y \mid x) = \frac{p(y)p(x \mid y)}{p(x)}.$$

Hence, under a uniform prior on the labels, i.e., $p(y = 1) = p(y = -1) = 1/2$, we have that $\widehat{f}$ is also equal to the *maximum-likelihood estimator* (MLE):

$$\widehat{f}(x) = \arg\max_{y \in \mathsf{Y}} p(x \mid y).$$

2 Perceptron

We now turn to answer the second question (b), what algorithm do we use to learn the representation of a decision function $f : \mathsf{X} \mapsto \mathsf{Y}$. We will start from a very simple prototypical algorithm, called the Perceptron, and understand its various properties. While the Perceptron is a very simple model, many interesting properties will arise that will inform our later study.

The Perceptron starts by making a modeling assuming regarding the form of the predictor $\widehat{f}$:

$$f_w(x) = \operatorname{sgn}\{\langle w, x \rangle\}, \quad w \in \mathbb{R}^d.$$

Learning here then refers to learning the weights w which parameterize the separating hyperplane. The Perceptron prescribes the following procedure to learn these weights.

We will now study the behavior of the Perceptron on *linearly separable* data.

Definition 2.1. *A dataset* $S_n = \{(x_1, y_1), \ldots, (x_n, y_n)\} \subset \mathsf{Z}$ *is* linearly separable *if there exists a non-zero* $w \in \mathbb{R}^d$ *such that:*

$$\forall i \in \{1, \ldots, n\}, \quad y_i = \operatorname{sgn}\{\langle w, x_i \rangle\}.$$

Such a non-zero $w \in \mathbb{R}^d$ *satisfying the above condition is called a* linear separator.

Algorithm 1 The Perceptron

Input: Dataset $S_n = \{(x_1, y_1), \ldots, (x_n, y_n)\}$.

Set $w = 0$.
while `true` **do**
  Let $\mathcal{I}$ be a (possibly random) permutation of $\{1, \ldots, n\}$.
  Set `mistake = false`.
  for $i \in \mathcal{I}$ **do**
    if $y_i \langle w, x_i \rangle \leqslant 0$ **then**
      Set $w \leftarrow w + y_i x_i$.
      Set `mistake = true`.
    end if
  end for
  if `not mistake` **then**
    return w.
  end if
end while

Next, we will define the notion of a *margin*, which quantifies to what degree a dataset is linearly seperable. Before we do this, let S be a subset of Euclidean space. The distance function $\operatorname{dist}(x, S)$ is defined as:

$$\operatorname{dist}(x, S) := \inf\{\|x - z\| \mid z \in S\}.$$

Also, for a vector $w \in \mathbb{R}^d$, we define its hyperplane H_w as:

$$H_w := \{x \in \mathbb{R}^d \mid \langle x, w \rangle = 0\}.$$

For any $w \neq 0$, the distance from x to H_w is:

$$\operatorname{dist}(x, H_w) = \frac{|\langle x, w \rangle|}{\|w\|}. \tag{2.1}$$

Indeed, the orthogonal projection of x onto H_w is $z_\star = P_w^\perp x = x - \frac{\langle x, w \rangle}{\|w\|^2} w$, which gives the upper bound $\operatorname{dist}(x, H_w) \leqslant \|x - z_\star\| = |\langle x, w \rangle| / \|w\|$. Conversely, we have for any $z \in H_w$,

$$\|x - z\|^2 = \|P_w^\perp x - z + P_w x\|^2 = \|P_w^\perp x - z\|^2 + \|P_w x\|^2 \geqslant \|P_w x\|^2 = \frac{|\langle w, x \rangle|^2}{\|w\|^2} \implies \operatorname{dist}(x, H_w) \geqslant \frac{|\langle w, x \rangle|}{\|w\|}.$$

We now have the requisite notation to define the notion of margin.

Definition 2.2. *Let S_n be a linearly separable dataset (cf. Definition 2.1), and let $W \subset \mathbb{R}^d$ denote the set of linear separators of S_n. Given $w \in W$, the* margin of w, *denoted $\gamma(w, S_n)$, is defined as:*

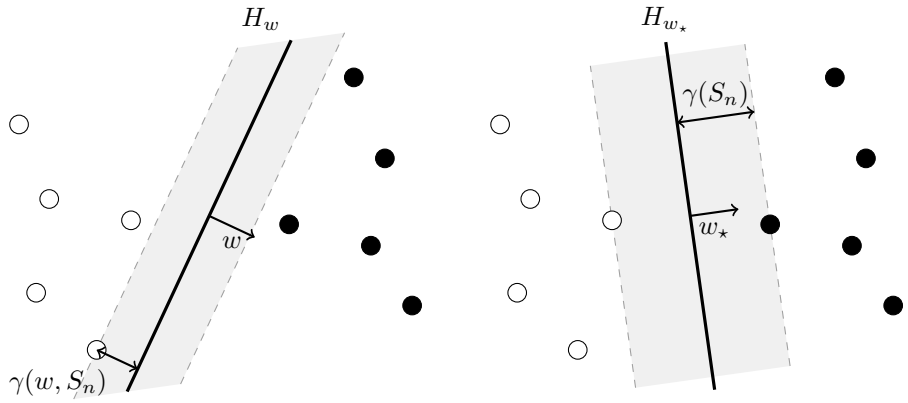
$$\gamma(w, S_n) := \min_{i \in \{1, \ldots, n\}} \operatorname{dist}(x_i, H_w). \tag{2.2}$$

Furthermore, the margin $\gamma(S_n)$ *is defined as the maximum margin achievable by any linear separator $w \in W$:*

$$\gamma(S_n) := \sup_{w \in W} \gamma(w, S_n) = \sup_{w \in W} \min_{i \in \{1, \ldots, n\}} \operatorname{dist}(x_i, H_w). \tag{2.3}$$

A max-margin *separator is any linear separator $w_\star \in W$ such that $\gamma(w_\star, S_n) = \gamma(S_n)$.*

The idea behind margin is to assign a quantity to a linear separator which measures how "robust" it is; see Figure 1.



(a) Non-max-margin separator **(b) Max-margin separator**

Figure 1: Two separators of the same training sample. The separator H_w on the left is valid but has a smaller margin $\gamma(w, S_n)$. The max-margin separator $H_{w_\star}$ on the right achieves $\gamma(S_n)$. In each panel, the shaded strip contains no training examples.

2.1 Mistake bound

With this notion of margin, we are able to prove our first non-trivial result regarding Perceptron, which is its well-known *mistake bound*. While this is a classic result, we will follow the derivations based on the presentation in Hardt and Recht [2022, Ch. 3].

To start, we will define a sequences of weights $w_0, w_1, \ldots$ recursively, which captures the main Perceptron update. Let $i_0, i_1, \ldots$ denote an *arbitrary* sequence of indices in $\{1, \ldots, n\}$ (meaning, these indices could be repeating, etc.). Our sequence $w_0, w_1, \ldots$ is defined as:

$$w_{t+1} = w_t + y_{i_t} x_{i_t} \mathbf{1}\{y_{i_t} \langle w_t, x_{i_t} \rangle \leqslant 0\}, \quad w_0 = 0. \tag{2.4}$$

Note that our notation makes implicit the dependence of the iterates $\{w_t\}$ on the indices $\{i_t\}$. In the sequel, it will be clear from context how the indices $\{i_t\}$ are specified.

Lemma 2.3 (Perceptron mistake bound)**.** *Let S_n be linearly separable. Consider the Perceptron sequence $\{w_t\}_{t \geqslant 0}$ defined in Equation (2.4). Let $m_t := \mathbf{1}\{y_{i_t} \langle w_t, x_{i_t} \rangle \leqslant 0\}$ denote the indicator of whether a mistake occured at time t, and let $M_t := \sum_{s=0}^{t-1} m_s$ denote the culmulative number of mistakes made up to time t. We have that for all $t \in \mathbb{N}$:*

$$M_t \leqslant \frac{\max_{i \in \{1,\ldots,n\}} \|x_i\|^2}{\gamma^2(S_n)}. \tag{2.5}$$

Before proceeding to the proof, it is worth reflecting on the nature of this result. It is quite remarkable in that it is *dimension independent*. Furthermore, since the RHS is independent of t, this result implies there exists a t such that for all $t' \geqslant t$, we have $m_{t'} = 0$ (why?). In other words, the Perceptron eventually stops making mistakes. Furthermore, since this result holds for *any* sequence $i_0, i_1, \ldots$, by setting the sequence $\{i_t\}$ to be repeated copies of $\{1, \ldots, n\}$, this ensures that eventually the Perceptron indeed learns a linear separator (why?).

Proof of Lemma 2.3. Put $B := \max_{i \in \{1,\ldots n\}} \|x_i\|^2$. Fix a t and suppose that $m_t = 1$. Then, by expanding the square:

$$\begin{aligned} \|w_{t+1}\|^2 &= \|w_t + y_{i_t} x_{i_t}\|^2 = \|w_t\|^2 + 2 y_{i_t} \langle w_t, x_{i_t} \rangle + \|x_{i_t}\|^2 \\ &\leqslant \|w_t\|^2 + 2 y_{i_t} \langle w_t, x_{i_t} \rangle + B \leqslant \|w_t\|^2 + B. \end{aligned}$$

The key here is the last inequality, which is a consequence of $m_t = 1$, or equivalently $y_{i_t}\langle w_t, x_{i_t}\rangle \leqslant 0$. On the other hand, if $m_t = 0$, then $\|w_{t+1}\|^2 = \|w_t\|^2$ by definition. Hence, for every t:

$$\|w_{t+1}\|^2 \leqslant \|w_t\|^2 + Bm_t.$$

Unrolling this recursion to $t = 0$ and using the definition of M_t yields for every t:

$$\|w_t\|^2 \leqslant B\sum_{s=0}^{t-1} m_s = BM_t.$$

Next, let $w_\star$ denote a unit norm max-margin linear separator, i.e., a vector such that (cf. Equation (2.3)):

$$\gamma(S_n) = \min_{1\leqslant i\leqslant n} |\langle x_i, w_\star\rangle|.$$

Now, suppose that $m_t = 1$, and hence, since $w_\star$ linearly separates S_n,

$$\langle w_\star, w_{t+1} - w_t\rangle = y_{i_t}\langle w_\star, x_{i_t}\rangle = |\langle w_\star, x_{i_t}\rangle| \geqslant \gamma(S_n).$$

Next, we use a *telescoping sum*, which is a trick often used in optimization proofs. The telescoping sum simply states (recall that $w_0 = 0$):

$$w_t = \sum_{k=1}^{t}(w_k - w_{k-1}) = \sum_{k=1}^{t}(w_k - w_{k-1})m_{k-1}.$$

While this may appear tautological, it is actually quite useful, since:

$$\|w_t\| \geqslant \langle w_\star, w_t\rangle = \sum_{k=1}^{t}\langle w_\star, w_k - w_{k-1}\rangle m_{k-1} \geqslant \gamma(S_n)\sum_{k=1}^{t} m_{k-1} = \gamma(S_n)M_t.$$

Above, the first inequality holds by Cauchy-Schwarz (recall that $w_\star$ is unit norm). We now have upper and lower bounds on $\|w_t\|$, from which we conclude:

$$\gamma^2(S_n)M_t^2 \leqslant \|w_t\|^2 \leqslant BM_t \implies M_t \leqslant \frac{B}{\gamma^2(S_n)}.$$

□

Corollary 2.4. *Suppose Algorithm 1 is called with a linearly separable dataset S_n. Let B bound the covariate norm, i.e., $B = \max_{i\in\{1,\ldots,n\}}\|x_i\|^2$. Then, Algorithm 1 terminates with a solution $w \in \mathbb{R}^d$, which linearly separates S_n, in at most $\lceil B/\gamma^2(S_n) + 1\rceil$ passes through S_n.*

Proof. Suppose that Algorithm 1 does not terminate on S_n in the first $\lceil B/\gamma^2(S_n) + 1\rceil$ passes. This means that for index $t = n\lceil B/\gamma^2(S_n) + 1\rceil$, we must have $\lceil B/\gamma^2(S_n) + 1\rceil \leqslant M_t$. But from Lemma 2.3, we have $M_t \leqslant B/\gamma^2(S_n)$, which yields a contradiction. □

2.2 Generalization bound

The mistake bound from Section 2.1 gives us a way to prove a generalization bound about Perceptron, on *unseen* instances. We will utilize a clever technique known as leave-one-out analysis. We need one more piece of notation before we proceed. Given a linearly separable dataset S_n, we let $w(S_n)$ denote the limiting value of the Perceptron sequence $\{w_t\}$ defined in Equation (2.4), where we fix the indices $\{i_t\}$ to repeatedly cycle through $\{1,\ldots,n\}$, i.e., $i_t = (t \mod n) + 1$. Note that $w(S_n)$ is well-defined by Corollary 2.4. For what follows, we will assume that the distribution $\mathcal{D}$ is linearly separable, meaning that almost surely, for any $n \in \mathbb{N}_+$, S_n sampled i.i.d. from $\mathcal{D}$ is linearly separable.

Lemma 2.5 (Perceptron generalization bound)**.** *Suppose that $\mathcal{D}$ is linearly separable. Also, suppose that $\|x\|^2 \leqslant B$ almost surely when $x \sim p(x)$. Then, letting $(x, y) \sim \mathcal{D}$ be drawn independently from S_n,*

$$\mathbb{P}\{y\langle w(S_n), x\rangle \leqslant 0\} \leqslant \frac{B}{n+1}\mathbb{E}\left[\frac{1}{\gamma^2(S_{n+1})}\right]. \tag{2.6}$$

Note that the probability on the LHS above is taken jointly over (S_n, x, y).

Proof. We will construct an equivalent probability space as follows. Let $S_{n+1} = \{z_1, \ldots, z_{n+1}\}$ be $n+1$ i.i.d. draws from $\mathcal{D}$, and let $S_{n+1}^{-k} := \{z_1, \ldots, z_{k-1}, z_{k+1}, \ldots z_{n+1}\}$ be the dataset where the k-th point is left out. Note that by exchangability, for any k,

$$\mathbb{P}\{y\langle w(S_n), x\rangle \leqslant 0\} = \mathbb{P}\{y_k\langle w(S_{n+1}^{-k}), x_k\rangle \leqslant 0\}.$$

Hence we can average over all indices to conclude:

$$\mathbb{P}\{y\langle w(S_n), x\rangle \leqslant 0\} = \frac{1}{n+1}\sum_{k=1}^{n+1}\mathbb{P}\{y_k\langle w(S_{n+1}^{-k}), x_k\rangle \leqslant 0\}.$$

(This type of relabeling argument is common to leave-one-out analyses, and we will see it come up again later when we study algorithmic stability.)

Now, let $I_m \subset \{1, \ldots, n+1\}$ denote the indices for which the Perceptron on S_{n+1} made a mistake at any point during execution, i.e.,

$$I_m := \{(t \mod (n+1)) + 1 \mid m_t = 1,\, t \in \mathbb{N}\}.$$

Let $k \in \{1, \ldots, n+1\}$, and suppose $k \notin I_m$. Then, we have the key equality:

$$w(S_{n+1}^{-k}) = w(S_{n+1}).$$

It is worth taking a moment to think why this equality is true. If $k \notin I_m$, then this means that each time Perceptron on S_{n+1} cycles through the index k, it makes no update. Hence, it is equivalent to running Perceptron on S_{n+1}^{-k}, where the k-th datapoint is altogether removed.

Now here is the final trick. Again let $k \notin I_m$. Then we have:

$$0 < y_k\langle w(S_{n+1}), x_k\rangle = y_k\langle w(S_{n+1}^{-k}), x_k\rangle.$$

(Check your understanding: why is the first inequality is true?) Therefore, chaining these arguments together and applying the mistake bound from Lemma 2.3,

$$\begin{aligned}\sum_{k=1}^{n+1}\mathbf{1}\{y_k\langle w(S_{n+1}^{-k}), x_k\rangle \leqslant 0\} &= \sum_{k\in I_m}\mathbf{1}\{y_k\langle w(S_{n+1}^{-k}), x_k\rangle \leqslant 0\} + \sum_{k\notin I_m}\mathbf{1}\{y_k\langle w(S_{n+1}^{-k}), x_k\rangle \leqslant 0\} \\ &= \sum_{k\in I_m}\mathbf{1}\{y_k\langle w(S_{n+1}^{-k}), x_k\rangle \leqslant 0\} \leqslant |I_m| \leqslant \frac{B}{\gamma^2(S_{n+1})}.\end{aligned}$$

The result now follows by taking expectations on both sides of the above inequality. □

Remark 2.6. Recall in the definition of $w(S_n)$, we specifically fixed the indices $\{i_t\}$ to repeatedly cycle through $\{1, \ldots, n\}$. Where in the proof of Lemma 2.5 did we use this assumption? What modifications to how the sequence $\{i_t\}$ is generated can you think of, which allow the proof to still go through?

Remark 2.7. Lemma 2.5 states that the margin mistake error $\mathbb{P}\{y\langle w(S_n), x\rangle \leqslant 0\}$ decreases at a rate of $O(1/n)$, assuming that $\sup_{n\in\mathbb{N}_+}\mathbb{E}[\gamma(S_n)^{-2}] < \infty$. In the literature, this is referred to as a *fast rate*, since it is faster than the usual $O(1/\sqrt{n})$ rate which arises out of typical concentration of measure arguments (which we will see in a few lectures). This fast-rate is not to be taken for granted: obtaining a fast-rate in general settings typically involves a much more non-trivial argument.

Interpreting the margin mistake bound. How does the margin mistake bound from Lemma 2.5 relate to the zero-one loss $\mathbb{P}\{y \neq \operatorname{sgn}(\langle w(S_n), x\rangle)\}$? Let us abbreviate $w = w(S_n)$. We can upper bound the zero-one loss by the margin mistake bound via the following argument:

$$\begin{aligned}\{y \neq \operatorname{sgn}(\langle w, x\rangle)\} &= \{y = 1, \langle w, x\rangle < 0\} \cup \{y = -1, \langle w, x\rangle \geqslant 0\} \\ &= \{y = 1, y\langle w, x\rangle < 0\} \cup \{y = -1, y\langle w, x\rangle \leqslant 0\} \\ &\subset \{y\langle w, x\rangle \leqslant 0\}.\end{aligned}$$

(Note the asymmetry in the cases is due to the tie-breaking choice made in our definition of the sgn function.) Hence by monotonicity of probability combined with Lemma 2.5:

$$\mathbb{P}\{y \neq \operatorname{sgn}(\langle w(S_n), x\rangle)\} \leqslant \mathbb{P}\{y\langle w(S_n), x\rangle \leqslant 0\} \leqslant \frac{B}{n+1}\mathbb{E}\left[\frac{1}{\gamma^2(S_{n+1})}\right].$$

References

Moritz Hardt and Benjamin Recht. *Patterns, predictions, and actions: Foundations of machine learning.* Princeton University Press, 2022.